

SMART COAL GOVERNANCE PLATFORM

Detailed Solution and Implementation Blueprint

Purpose

Convert fragmented mine governance records into a traceable workflow that detects risk, assigns action, verifies evidence, and closes issues.

Core outcome	Implementation focus
Single operational view	Mine, subsidiary, compliance, inspection, contractor and corrective-action data in one platform.
Field to control room	Geo-tagged inspection capture with offline storage and controlled synchronization.
Risk-led governance	Mine risk score, recurring violation detection and anomaly detection with explicit reasons.
Closed-loop accountability	Every issue has owner, deadline, evidence, verification and audit history.
Hackathon-ready scope	Working MVP first; OCR, chatbot, blockchain and advanced integrations are modular extensions.

Design rule: every feature must produce a visible governance outcome.

Contents

- 1. Problem Interpretation
- 2. Proposed Solution
- 3. Target Users and Permissions
- 4. MVP Scope and Feature Priorities
- 5. Key Implementable USPs
- 6. End-to-End Operating Workflow
- 7. System Architecture
- 8. Recommended Technology Stack
- 9. Data Model
- 10. Core API Design
- 11. AI and Analytics Design
- 12. Offline Mobile Design
- 13. GIS Design
- 14. OCR Document Pipeline
- 15. Notifications, Escalation and Auditability
- 16. Security and Governance Controls
- 17. Detailed Implementation Phases
- 18. Team Allocation and Effort
- 19. Hackathon Execution Plans
- 20. Testing and Acceptance Criteria
- 21. Risks and Mitigations
- 22. Demo Data Strategy
- 23. 7 to 10 Minute Demo Storyboard
- 24. Production Evolution
- 25. Final Build Order
- 26. Compact Claude Second-Pass Prompt

1. Problem Interpretation

The problem is not the absence of individual software tools. The problem is the lack of a single operational chain connecting field evidence to management action. The proposed platform should therefore be designed around traceability rather than around a collection of disconnected dashboards.

Current failure	What the platform must change
Records are spread across spreadsheets, PDFs and separate systems.	Create one canonical record for each mine, compliance obligation, inspection, observation and corrective action.
Field observations arrive late and may lack proof of place and time.	Capture GPS, timestamp, inspector identity and evidence at the point of observation.
Issues are recorded but closure is difficult to verify.	Force an owner, due date, evidence upload and verifier before closure.
Managers see counts instead of reasons.	Show risk scores with the exact contributing factors and trend direction.
Connectivity can be unreliable at mine sites.	Use offline inspection capture with queued synchronization and conflict handling.
Regulatory reporting is repetitive.	Generate structured reports from already captured records instead of retyping data.

2. Proposed Solution

The proposed product is a role-based web and mobile platform with one operational database and a small AI analytics layer. The platform is organized around five linked records: requirement, observation, action, evidence and verification.

Requirement -> Inspection / Evidence -> Observation or Violation -> Corrective Action -> Evidence -> Verification -> Closure

The central dashboard should not merely report that a mine has 18 open items. It should expose why the mine is high risk, which items are late, who owns them, and what changed since the previous reporting period.

Module	Primary job	Must support in MVP
Mine command dashboard	Prioritize management attention.	Compliance %, open violations, overdue actions, risk score, trends, recent field activity.
Compliance register	Track obligations and deadlines.	Requirement, frequency, owner, due date, evidence, status, reminders.
Field inspection	Capture reliable on-site evidence.	Inspection form, GPS, timestamp, photo, severity, offline save, sync.
Corrective action	Close the loop after a finding.	Assignment, deadline, evidence, verifier, closure state.
Risk engine	Rank mines and issues.	Weighted score, recurring failure signal, anomaly signal, explainability.
GIS layer	Show where issues occur.	Mine markers, observation markers, severity filter, map drill-down.
Notification engine	Push work to the right person.	Due-soon, overdue, critical risk and escalation alerts.

3. Target Users and Permissions

Role	Read	Create	Approve / Verify	Key screens
Field Inspector	Assigned mines and open tasks	Inspections, observations, photos	No	Mobile inspection, sync queue, own tasks
Safety / Compliance Officer	Mine records and inspection history	Compliance records, corrective actions	Verify closure	Compliance register, action queue
Mine Manager	All records for assigned mine	Management comments, action assignments	Approve closure where allowed	Mine dashboard, risk view, action board

Corporate Manager	All subsidiaries / mines	Escalation rules, review notes	Executive review	Enterprise dashboard, mine comparison
Regulatory Reviewer	Submitted regulatory records	Inspection review notes where permitted	Regulatory acceptance where applicable	Compliance status, evidence, audit trail
Contractor Manager	Contractor records and worker compliance	Contractor updates, training records	Verify contractor submissions	Contractor dashboard
System Admin	System configuration	Users, roles, reference data	Configuration approval	Admin console

RBAC rule: a user must never gain edit access merely because a record is visible on a dashboard. Write permissions are tied to role, organization and mine scope.

4. MVP Scope and Feature Priorities

Priority	Feature	Reason
P0	Login + RBAC	Required to demonstrate role-specific governance.
P0	Compliance register	Directly matches the statutory tracking requirement.
P0	Field inspection with GPS + photo	Provides the field-to-system proof point.
P0	Corrective action workflow	Shows accountability and closure rather than passive reporting.
P0	Mine risk score	Turns raw records into prioritization.
P0	Management dashboard	Makes the data useful to decision makers.
P0	GIS issue map	Demonstrates spatial visibility of field problems.
P0	Alerts and escalation	Converts detected risk into action.
P1	OCR document extraction	Reduces manual entry and provides an AI/document angle.
P1	Contractor compliance	Useful extension once core workflow is stable.
P1	Production anomaly detection	Strengthens analytics story with operational signals.
P2	Natural language governance assistant	Useful for query and summary, but not needed for core workflow.
P2	Blockchain audit layer	Only add after proving database audit logs are insufficient.
P3	IoT sensor ingestion	Production-scale extension; do not block the MVP on it.

MVP boundary: the demo must work from field observation to verified closure before adding more modules.

5. Key Implementable USPs

The following USPs are implementable in a student prototype and are tied to specific product behavior. They avoid claims that depend on proprietary government systems or large hidden datasets.

1. Governance Risk Cockpit: Each mine receives a 0 to 100 risk score built from overdue compliance, violation severity, recurrence, corrective-action delay and selected operational anomalies. Every score is accompanied by a top-five reason list.

2. Violation Recurrence Fingerprint: Normalize observation category + mine zone + contractor + severity into a fingerprint. Detect repeated findings across inspections and mark the pattern as recurring when configurable thresholds are crossed.

3. Evidence Integrity Bundle: Every field finding stores GPS, timestamp, user identity, device time, photo hash and server receipt time. This creates a tamper-evident evidence trail without requiring blockchain for the MVP.

4. Offline Field Queue with Safe Sync: Inspections can be completed with no network. Each local record gets a temporary UUID, remains in a sync queue, retries automatically, and shows a clear synced / pending / conflict state.

5. Corrective Action SLA Ladder: A configurable timer changes state from due soon to overdue to escalated. The owner, supervisor and corporate reviewer receive different alerts based on severity and age.

6. Compliance Heatmap by Mine Zone: The GIS view aggregates findings by mine zone. A manager can filter by category, severity and date and immediately see concentration areas instead of only seeing a mine-wide count.

7. AI Report-to-Record Converter: OCR extracts date, mine, inspection type, finding, due date and named owner from a sample PDF or scanned form, then asks a human reviewer to confirm low-confidence fields before saving.

8. Risk Change Narrator: Instead of only showing a new score, the dashboard compares the current period with the previous period and generates a concise explanation such as: risk increased because three high-severity findings remained open and two compliance items became overdue.

9. Contractor Compliance Fingerprint: Calculate a contractor score using training completion, active violations, repeat observations and overdue actions. Compare contractors operating inside the same mine.

10. One-Tap Escalation Trace: Every alert carries a link to the issue, current owner, deadline, previous escalation and latest evidence. This removes the need to search across email threads.

6. End-to-End Operating Workflow

A. Compliance cycle

Requirement created -> due date tracked -> reminder -> submission -> evidence check -> approval -> next cycle

B. Field issue cycle

Inspector creates inspection -> observation captured -> severity assigned -> corrective action created -> owner notified -> evidence uploaded -> verifier checks -> closed

C. Risk cycle

Records aggregated -> features calculated -> risk score updated -> top drivers generated -> dashboard highlight -> alert if threshold crossed

D. Offline cycle

Form completed -> local encrypted store -> sync queue -> network check -> API submission -> server ID returned -> local record marked synced

7. System Architecture

Web app / Mobile app

|

v

API layer (FastAPI)

|---- Auth / RBAC

|---- Compliance service

|---- Inspection service

|---- Corrective action service

|---- Dashboard query service

|---- Notification service

|

+---- PostgreSQL + PostGIS

+---- Object storage (photos / documents)

+---- Background worker (scheduler, OCR, AI scoring)

+---- ML service / Python module

+---- Audit log store / immutable append-only tables

Component	Responsibility	Recommended implementation
Web client	Management, compliance and regulatory views	React + TypeScript + Vite or Next.js
Mobile client	Field inspection and offline capture	React Native with local SQLite or

		WatermelonDB
API	Business logic and authorization	FastAPI + Pydantic + SQLAlchemy
Database	Transactional and spatial records	PostgreSQL + PostGIS
Object storage	Photos, PDFs and evidence	S3-compatible bucket
Worker	Scheduled alerts, OCR and scoring	Celery or lightweight async worker with Redis
AI module	Risk, recurrence, anomaly and explanation	Python + pandas + scikit-learn
Map	Mine and issue visualization	Leaflet + OpenStreetMap tiles
Notifications	In-app and email for demo	Firestore Cloud Messaging + SMTP or provider API
Deployment	Fast prototype hosting	Docker Compose locally; cloud deployment on Render / Railway / AWS as needed

Architecture decision: use a modular monolith for the prototype. Separate services are not required until scale or team boundaries justify them.

8. Recommended Technology Stack

Area	Choice	Reason	Avoid initially
Web	React + TypeScript	Fast component development, strong dashboard ecosystem.	Micro-frontends
Mobile	React Native	Shares TypeScript concepts with web; practical offline storage.	Native Android + iOS separate codebases
Backend	FastAPI	Clear validation, fast development, strong Python AI integration.	Multiple backend frameworks
DB	PostgreSQL + PostGIS	Relational integrity plus geospatial queries.	Using MongoDB for transactional workflow
AI	scikit-learn	Works with small synthetic datasets and is explainable.	Deep neural network training without data
OCR	PaddleOCR or Tesseract	Local prototype, no proprietary dependency.	Building OCR from scratch
Auth	JWT + refresh token	Simple role based access for prototype.	Enterprise SSO implementation unless required
Maps	Leaflet + OSM	Low setup cost, sufficient for demo.	Custom GIS engine
Background work	Celery + Redis or FastAPI background tasks	Supports scheduled reminders and scoring.	Kafka for a hackathon
Deploy	Docker	Reproducible local and cloud environment.	Kubernetes

9. Data Model

Table	Key fields	Relationships
organizations	id, name, type, parent_id	Self hierarchy for corporate -> subsidiary.
mines	id, org_id, name, code, latitude, longitude, status	Belongs to organization.
users	id, org_id, role_id, name, email, active	Belongs to organization; role controlled.
roles	id, name	Used by users and permission map.
compliance_requirements	id, category, title, frequency, severity, owner_role	Reference definition.
compliance_records	id, mine_id, requirement_id, due_date, status, submitted_at	Instance of a requirement for a mine.
inspections	id, mine_id, inspector_id, started_at,	Parent for observations.

	ended_at, latitude, longitude, sync_state	
observations	id, inspection_id, category, severity, description, location_geom, photo_id	May create corrective action.
violations	id, observation_id, type, status, recurrence_key	Normalized governance issue.
corrective_actions	id, violation_id, owner_id, due_date, status, verified_by	Closed only after verification.
evidence	id, entity_type, entity_id, file_url, sha256, captured_at	Linked to inspection, action or compliance record.
contractors	id, mine_id, name, contract_start, contract_end	Contract scope.
contractor_workers	id, contractor_id, worker_code, training_status	Training and attendance extensions.
production_records	id, mine_id, record_date, output_tonnes	Used for anomaly signals.
environment_records	id, mine_id, record_date, metric, value, unit	Used for threshold and anomaly checks.
risk_scores	id, mine_id, score, band, model_version, generated_at	Stores explainable output.
notifications	id, user_id, type, entity_id, status, sent_at	Delivery tracking.
audit_logs	id, actor_id, entity_type, entity_id, action, before_json, after_json, created_at	Append-only change history.

Key design choice: use immutable IDs and soft deletion for governance records. A closed compliance or corrective-action record should not be physically deleted by normal users.

10. Core API Design

Method	Endpoint	Purpose	Role
POST	/auth/login	Login and token issue	All
GET	/mines	List accessible mines	All authorized
GET	/dashboard/summary?mine_id=	Management KPI summary	Manager / Corporate
GET	/compliance?mine_id=	List compliance records	Compliance / Manager
POST	/compliance	Create compliance record	Compliance
PATCH	/compliance/{id}	Update status / evidence metadata	Compliance
POST	/inspections	Create inspection	Inspector
POST	/inspections/{id}/observations	Add observation	Inspector
POST	/inspections/sync	Upload offline records	Inspector
GET	/violations?status=open	Open issue queue	Authorized
POST	/corrective-actions	Create action	Officer / Manager
PATCH	/corrective-actions/{id}	Update action progress	Owner / Manager
POST	/corrective-actions/{id}/verify	Verify closure	Verifier
GET	/risk/mines	List risk ranked mines	Manager / Corporate
GET	/risk/mines/{id}	Risk detail and reasons	Manager / Corporate
GET	/gis/observations	Geo-filtered observations	Authorized
POST	/documents	Upload document	Authorized
POST	/documents/{id}/extract	Run OCR extraction	Compliance
GET	/audit/{entity}/{id}	Entity audit history	Authorized

11. AI and Analytics Design

Use a layered intelligence design. Rule logic handles deterministic deadlines and severity. Machine learning handles patterns that are difficult to encode as fixed rules. Generative AI is optional and should explain or summarize records, not decide compliance on its own.

Capability	Input	Method	Output	Evaluation
------------	-------	--------	--------	------------

Compliance risk	Overdue %, days overdue, severity, recurrence	Weighted score first; later logistic/XGBoost if data exists	0 to 100 score + reasons	Compare score against labeled risk scenarios
Recurring violation detection	Category, zone, contractor, date, severity	Fingerprint + frequency window	Recurring flag + recurrence count	Precision on seeded repeat cases
Production anomaly	Daily production series	Rolling z-score / Isolation Forest	Anomaly score + date	Known seeded anomalies detected
Risk trend	Historical scores and components	Period-over-period comparison	Increasing / stable / decreasing	Correct trend direction
Document extraction	OCR text	Pattern extraction + optional LLM normalization	Structured fields + confidence	Field-level extraction accuracy
Report summary	Inspection / compliance records	Template or LLM summary	Management summary	Human review for factual consistency

11.1 Mine Risk Score

$$\text{BaseScore} = 0.30 * \text{Compliance} + 0.25 * \text{Violations} + 0.20 * \text{CorrectiveAction} + 0.15 * \text{Safety} + 0.10 * \text{Operations}$$

Normalize each component to 0 to 100. Example: Compliance risk rises with overdue percentage and severity of late items. Violation risk rises with open severity-weighted findings and recurrence. Corrective-action risk rises with overdue actions and age of overdue actions. Operations can include production anomaly rate in the prototype.

Band	Score	Meaning	Action
Low	0 to 29	Normal governance load	Routine review
Watch	30 to 49	Some indicators worsening	Manager review
High	50 to 69	Material unresolved risk	Corrective review within defined SLA
Critical	70 to 100	Multiple severe signals	Escalation and priority inspection

12. Offline Mobile Design

Local form -> local SQLite record -> temp UUID -> attachment queue -> connectivity check -> sync POST -> server validation -> server UUID -> mark synced

Concern	Implementation
Local storage	SQLite records for drafts and submitted-but-not-synced inspections.
Temp IDs	UUID generated on device; server stores idempotency key.
Attachments	Compressed photos stored locally until upload succeeds.
Retries	Exponential retry with capped attempts; clear failed state.
Conflicts	Server is authoritative for records already accepted. New observations with different UUIDs are merged rather than overwritten.
Timestamp	Store device capture time and server receipt time separately.
GPS	Capture latitude, longitude, accuracy and capture timestamp.
Sync indicator	Show counts for pending, syncing, synced and failed records.
Security	Encrypt local database where platform capability is available and protect device session with PIN / biometric as an extension.

13. GIS Design

Map feature	Implementation
Mine markers	Store latitude/longitude in mines table and render clustered markers.

Observation points	PostGIS point geometry from field GPS.
Severity filter	Filter red / amber / green observations.
Date filter	Show new, unresolved and historical findings.
Zone concentration	Aggregate observation counts into a grid or defined mine zones for heatmap.
Drill-down	Click marker -> issue summary -> evidence -> corrective action status.

14. OCR Document Pipeline

Upload PDF/image -> object storage -> OCR -> text normalization -> field extraction -> confidence score -> human review -> save structured record -> audit log

Field	Example extraction	Validation
Mine	Korba Mine A	Must match known mine
Date	29-08-2026	Valid date
Observation	PPE non-compliance	Required text
Severity	High	Enum mapping
Due date	31-08-2026	Not earlier than issue date unless explicitly allowed
Owner	Safety Officer	Map to user/role

15. Notifications, Escalation and Auditability

The workflow should treat time as a governance input. Every corrective action has a due date and escalation clock. Thresholds are configurable rather than hard-coded to one regulatory process.

Trigger	Primary action	Escalation
7 days before due	Notify owner	None
2 days before due	Notify owner + supervisor	Optional manager copy
Due date reached	Set status overdue	Notify manager
7 days overdue	Escalate	Corporate compliance queue
Critical observation	Immediate alert	Mine manager + compliance lead
Repeated violation	Increase recurrence weight	Priority inspection recommendation

Audit event	Store
Create	Actor, timestamp, entity, initial values
Update	Changed fields, previous values, new values
Assign	Old owner, new owner, actor, timestamp
Evidence upload	File hash, uploader, time
Verify	Verifier, decision, timestamp, evidence reference
Close / reopen	State transition, reason, actor

16. Security and Governance Controls

Control	Prototype implementation	Production extension
Authentication	JWT access token + refresh token	Enterprise SSO / MFA
Authorization	RBAC + mine scope	Policy engine + organization hierarchy
API security	Pydantic validation, parameterized queries, rate limits	WAF, API gateway, advanced throttling
File security	Type/size validation, random object names, malware scan hook	Dedicated scanning pipeline
Audit	Append-only audit_logs	Centralized immutable audit service
Secrets	Environment variables / secret store	Cloud secret manager
Backup	Scheduled DB backup	Cross-region retention and restore testing
Privacy	Collect only required worker/field data	Retention and legal policy enforcement

17. Detailed Implementation Phases

Phase	Focus	Core tasks	Deliverable	Acceptance criterion	Effort
Phase 0	Scope, repository and acceptance baseline	Define MVP, create repo structure, seed data contract, wireframes, branching strategy.	Deliverable: approved backlog and demo path.	Acceptance: team can explain one complete workflow and every P0 feature has an owner.	8 to 12 hours
Phase 1	Architecture and database	Create ERD, PostgreSQL/PostGIS schema, migrations, seed data.	Deliverable: schema + migrations + sample data.	Acceptance: fresh developer can run DB and load demo dataset.	12 to 18 hours
Phase 2	Backend foundation + auth	FastAPI project, models, CRUD base, JWT, RBAC middleware, error handling, OpenAPI.	Deliverable: authenticated API.	Acceptance: role access is enforced for protected routes.	14 to 20 hours
Phase 3	Compliance module	Requirement records, due dates, statuses, reminders, dashboard aggregates.	Deliverable: compliance register.	Acceptance: overdue and due-soon states calculate correctly.	14 to 18 hours
Phase 4	Inspection + offline sync	Inspection schema, photo upload, GPS, local DB, sync queue and status.	Deliverable: field capture flow.	Acceptance: inspection can be created without network and sync after reconnect.	20 to 28 hours
Phase 5	Corrective action + escalation	Assignment, SLA timer, evidence, verification, state machine, alerts.	Deliverable: closed-loop workflow.	Acceptance: an issue can move from Open to Assigned to In Progress to Verification to Closed with audit events.	18 to 24 hours
Phase 6	Dashboard + GIS	Mine dashboard, risk tiles, issue table, map, filters.	Deliverable: management control room.	Acceptance: dashboard values match DB and map filters show corresponding records.	18 to 26 hours
Phase 7	AI/analytics	Risk score, recurrence fingerprint, seeded anomaly detection, explanations.	Deliverable: risk engine.	Acceptance: known seeded high-risk mine ranks above low-risk mines and reasons match input data.	18 to 26 hours
Phase 8	OCR + reports	Upload sample PDF, OCR extraction, confidence, human confirmation, export summary.	Deliverable: document digitization demo.	Acceptance: target fields extracted from sample documents with review step.	10 to 16 hours
Phase 9	Integration + polish	End-to-end integration, loaders, notifications, error states, mobile UX, performance fixes.	Deliverable: stable demo build.	Acceptance: full demo path runs from login to closure without manual DB edits.	16 to 24 hours
Phase 10	Testing + deployment + demo prep	Test suite, bug triage, Docker, cloud deployment, demo data reset, presentation script.	Deliverable: tagged release + demo environment.	Acceptance: clean deployment can be recreated and demo recovers from reset.	16 to 22 hours

17.1 Parallelization

- Backend foundation and UI wireframes can run in parallel after the schema is agreed.
- AI prototype can start with synthetic CSV data while backend APIs are under construction.
- GIS work can start once the mine and observation geometry fields are finalized.

- OCR can be built against a fixed sample document before production document storage is complete.

17.2 Critical Path

Scope -> Schema -> Auth/API -> Compliance + Inspection -> Corrective Action -> Dashboard -> Risk Engine -> Integrated Demo

18. Team Allocation and Effort

Person	Primary ownership	Secondary ownership	Definition of done
Person 1	Backend + DB	Security + deployment	All P0 APIs stable, schema migrates cleanly, auth works.
Person 2	Web dashboard	UX + reporting	Dashboard, compliance, action board and filters are usable.
Person 3	Mobile + GIS	Offline sync	Field capture, GPS, sync queue and map are working.
Person 4	AI/analytics	OCR + integration	Risk engine, recurrence, anomaly demo and OCR pipeline are reproducible.

If only 3 people are available: combine GIS with frontend, combine OCR with AI, and postpone contractor management and chatbot.

Workstream	Estimated hours	Priority
Backend + database	45 to 60	P0
Web dashboard	35 to 50	P0
Mobile + offline	30 to 40	P0
AI + analytics	20 to 30	P0
GIS	10 to 16	P0
OCR	10 to 16	P1
Auth + security	12 to 18	P0
Notifications	8 to 12	P0
Testing + deployment	20 to 28	P0
Demo + docs	12 to 18	P0

Prototype total: approximately 202 to 288 engineering hours. With four students working in parallel, the schedule is feasible only if the scope stays inside the P0 boundary.

19. Hackathon Execution Plans

Timebox	Build	Defer	Demo target
24 hours	Auth, seeded data, compliance, inspection, corrective action, dashboard, simple risk score.	Offline sync depth, OCR, contractor, advanced ML.	One mine generates issue -> risk -> action -> closure.
48 hours	All 24h items + offline sync + GIS + recurring violation detection + alerts.	Advanced OCR and production-grade ML.	Two mines compared; field issue shown on map; escalation demonstrated.
7 days	Full MVP + OCR + anomaly detection + polished analytics + stronger audit trail.	Blockchain and enterprise integrations.	10-minute story with multiple personas and resettable dataset.

20. Testing and Acceptance Criteria

Test	Scenario	Pass condition
Auth	Inspector calls manager-only endpoint	403 returned and no state changed.
Compliance	Due date passes current date	Record becomes overdue in dashboard.
Inspection	No internet	Inspection saved locally and visible in

		pending queue.
Sync	Network restored	Server receives one record; retry does not duplicate it.
Evidence	Upload invalid file	Request rejected and no DB evidence record created.
Corrective action	Owner submits evidence	Status moves to Verification, not Closed.
Verification	Authorized verifier closes issue	Close event added to audit log.
Recurrence	Seed same issue three times	Fingerprint flagged recurring.
Risk	Overdue and severe findings added	Score rises and top reasons match added signals.
GIS	Filter high severity	Only matching markers remain visible.
OCR	Sample report uploaded	Target fields appear with confidence and review status.
Audit	Update owner	Before and after values are visible to authorized users.

20.1 System-level acceptance criteria

- An inspector can create a field inspection with GPS, timestamp, description and photo.
- An inspection created offline remains available until synchronization succeeds.
- Every violation has an owner, deadline and status.
- Every closed corrective action points to evidence and a verifier.
- The dashboard shows mine risk with contributing factors, not only a single score.
- GIS shows the inspection location stored in the database.
- An overdue item creates a visible alert and escalation state.
- All key state transitions create audit records.
- No demo scenario requires editing production database rows by hand.
- Fresh deployment plus seed script produces the same demo environment.

21. Risks and Mitigations

Risk	Probability	Impact	Mitigation	Fallback
No real mine dataset	High	High	Use synthetic data with seeded patterns and document assumptions.	Use rule-based scoring only.
AI model lacks training data	High	High	Use weighted model + Isolation Forest on synthetic series.	Disable ML and retain deterministic risk score.
Offline sync consumes too much time	Medium	High	Implement one-way create-and-sync first; avoid complex edits offline.	Use PWA queue if native sync slips.
OCR quality poor	Medium	Medium	Use clean sample documents and confidence threshold.	Manual confirmation with highlighted extracted fields.
Scope creep	High	High	Freeze P0 backlog before coding.	Move non-critical modules out of demo.
Deployment instability	Medium	High	Dockerize early and test from clean machine.	Run demo locally on one laptop with seeded DB.
Map tile/API limit	Low	Medium	Use OSM and cache sample map state.	Render a static map screenshot for demo only as backup.
AI explanation is misleading	Medium	High	Show input factors and model version.	Use rule-based explanations only.

22. Demo Data Strategy

Dataset	Target volume	Seeded scenario
Mines	6	2 low risk, 2 medium, 1 high, 1 critical
Compliance records	120	Overdue cluster in critical mine

Inspections	80	High-severity findings concentrated in one zone
Violations	60	Recurring PPE and access-control pattern
Corrective actions	70	Mix of open, due soon, overdue and closed
Contractors	12	One contractor with elevated repeat findings
Production records	180 days	One seeded production drop anomaly
Environment records	180 days	One metric excursion for alert demo
Documents	10 sample files	At least 3 OCR-friendly reports

Create a reset script that restores the exact dataset used in the presentation. Demo reliability is more valuable than adding another feature.

23. 7 to 10 Minute Demo Storyboard

Minute	Action	What the judge should notice
0:00 to 1:00	Login as corporate manager	One login exposes organization-aware dashboard.
1:00 to 2:00	Open mine ranking	Critical mine is ranked first with visible reasons.
2:00 to 3:00	Drill into risk	Dashboard shows overdue compliance, recurring findings and action age.
3:00 to 4:30	Switch to inspector mobile	Create field observation with GPS and photo; show offline queue if available.
4:30 to 6:00	Issue enters corrective workflow	Owner, SLA deadline, evidence and escalation appear.
6:00 to 7:00	Open GIS	Observation appears at captured location; filter by severity.
7:00 to 8:00	Upload OCR report	Fields extracted; low-confidence field requires confirmation.
8:00 to 9:00	Verify corrective action	Evidence checked; issue closed; audit history visible.
9:00 to 10:00	Return to dashboard	Risk and open-action counts update, showing the closed loop.

24. Production Evolution

Prototype	Production evolution
Modular monolith	Split high-volume workloads only where profiling justifies it.
JWT auth	Enterprise identity, MFA and organization federation.
Single database	Partitioning, replicas, backups and high availability.
Simple worker	Queue-based background processing with monitoring.
Synthetic data	Validated domain dataset and governance-approved historical records.
Rule + classical ML	Model registry, drift checks, retraining pipeline and human review.
Audit table	Append-only event stream with stronger immutability controls or permissioned ledger if a governance policy requires it.
Demo notifications	Enterprise notification gateway and escalation policy management.
Manual compliance definitions	Versioned compliance catalog with legal review and effective dates.

25. Final Build Order

1. Freeze MVP scope
2. Create PostgreSQL/PostGIS schema + seed data
3. Implement JWT + RBAC
4. Implement compliance APIs + dashboard tiles
5. Implement inspection capture + photo/GPS
6. Implement corrective action state machine
7. Connect dashboard to live API
8. Add mine risk score + explanations
9. Add GIS view
10. Add reminders + escalation
11. Add offline synchronization
12. Add OCR
13. Add recurrence + production anomaly
14. Run acceptance test script
15. Dockerize + seed reset + rehearse demo

25.1 Build First

- Compliance register and overdue logic
- Field inspection with GPS and evidence
- Corrective action state machine
- Mine dashboard with risk factors
- Role-based authentication
- GIS observation map
- Alerts and escalation

25.2 Build Later

- OCR workflow
- Contractor fingerprint
- Production anomaly model
- Environment anomaly model
- Natural language assistant

25.3 Do Not Build in MVP

- Custom blockchain network
- Microservice mesh
- Kubernetes
- Deep learning model trained from scratch
- Full payroll or ERP functions
- Real government system integration without an approved API and test environment

26. Compact Claude Second-Pass Prompt

Use the following prompt after giving Claude this solution document. The goal is to force critique, improve the USPs and remove vague features before implementation.

Read the entire solution document before answering. Do not summarize it. Act as a senior product architect reviewing a hackathon build for an Indian coal-mining governance platform.

- 1) Identify the 5 highest-value implementation gaps, contradictions or over-scoped items.
- 2) Propose 5 to 8 additional USPs that are implementable by a 3 to 5 person student team using open-source tools and synthetic data. Every USP must name the exact user action, data input, system behavior and visible demo outcome. Reject ideas that depend on proprietary data, vague AI claims, blockchain for appearance, or unrealistic integrations.
- 3) For every recommended USP, state build effort, dependency, failure mode and MVP priority.
- 4) Replace generic phrases such as “real-time insights”, “enhanced transparency”, “AI-powered analytics” or “seamless integration”

- 5) Produce a revised P0/P1/P2 backlog and a final critical path.
- 6) Return concrete engineering decisions, not motivational language.

Constraint: never use em dashes. Use commas, parentheses or colons instead. Prefer specific nouns, field names, workflow states, API actions and measurable acceptance criteria.

Recommended product principle: detect -> explain -> assign -> act -> verify -> record.